Labo 07 — Réseau : faire communiquer des conteneurs

Théorie — comment trois conteneurs qui ne connaissent pas leurs adresses se trouvent par leur nom, ce que « publier un port » veut vraiment dire, et pourquoi `localhost` est le piège numéro un.

Objectifs

- Comprendre ce qu'un conteneur reçoit comme réseau, et ce que le mode rootless y change (`pasta`).
- Distinguer le réseau par défaut d'un réseau **user-defined**, et savoir pourquoi le second a un DNS.
- Lire et écrire `-p`, `-P`, `EXPOSE`, `--network`, `--network-alias`.
- Diagnostiquer les trois erreurs classiques : `localhost`, port non publié, nom non résolu.
- Monter la chaîne **web → api → db** par les noms, et comprendre la notion de *pod*.

---

1. Le problème : trois processus, trois îlots

Au labo 06, `db` tournait seul. Dès qu'on ajoute `api` et `web`, il faut que nginx trouve l'API, et que l'API trouve PostgreSQL. Or chaque conteneur est isolé : il a sa propre pile réseau, sa propre adresse, et **son propre `localhost`**. Dans le conteneur `api`, `localhost:5432` désigne l'API elle-même, pas la base. C'est l'erreur la plus fréquente de tous les débutants, et de pas mal d'anciens.

> **→ LINUX**
> Un **network namespace** donne à un processus sa propre pile réseau : ses interfaces, sa table de routage, ses règles de pare-feu, ses ports. Deux namespaces peuvent avoir chacun un port 8080 ouvert sans conflit. `lo` (127.0.0.1) existe dans chacun, et ne mène qu'à lui-même. Un conteneur, c'est entre autres un network namespace — c'est pour ça que `localhost` y est un cul-de-sac.

Les adresses IP n'aident pas davantage : attribuées à la création, elles changent quand un conteneur est recréé. On ne code jamais une IP de conteneur. On a besoin de **noms**.

2. Ce que reçoit un conteneur

Sur un Docker classique, le daemon root crée un **bridge** (`docker0`), branche chaque conteneur dessus par une paire d'interfaces virtuelles, et fait du NAT vers l'extérieur. Le conteneur a une IP privée (`172.17.0.x`) et sort sur Internet par l'adresse de l'hôte.

> **→ RÉSEAU**
> Un **bridge** est un commutateur logiciel : des interfaces branchées dessus forment un réseau local. Une paire **veth** est un câble virtuel dont une extrémité est dans le conteneur (`eth0`) et l'autre sur le bridge. Le **NAT** (*network address translation*) réécrit l'adresse source des paquets sortants en celle de l'hôte, pour que les réponses reviennent. Trois briques du noyau, assemblées par le moteur.

PODMAN
Un utilisateur ordinaire n'a pas le droit de créer un bridge ni de faire du NAT. En rootless, Podman confie donc le réseau par défaut à `pasta`, un programme en espace utilisateur qui *copie* l'adresse de l'hôte dans le conteneur : `ip addr` dans le conteneur montre la même IP que votre WSL, et `inspect` affiche une `IPAddress` vide. Les réseaux **bridge** restent possibles (`--network podman`, ou un réseau créé par vous) : ils vivent dans un namespace privé de Podman, et c'est `rootlessport` qui relaie les ports publiés — en masquant, au passage, l'adresse du client réel.

Réseau	Comment l'obtenir	Adresse du conteneur	DNS entre conteneurs
par défaut (pasta)	ne rien préciser	celle de l'hôte, copiée	non
`podman` (bridge)	`--network podman`	`10.88.0.x`	**non** (`dns_enabled: false`)
user-defined (bridge)	`podman network create stack` puis `--network stack`	`10.89.x.x`	**oui** (`aardvark-dns`)
host	`--network host`	celle de l'hôte, sans isolation	—
none	`--network none`	aucune, seulement `lo`	—

3. Le réseau user-defined et son DNS

C'est la règle d'or de ce labo : **une application = un réseau créé pour elle**. `podman network create stack` crée un bridge privé avec un sous-réseau (`10.89.0.0/24`) et, surtout, un **serveur DNS interne** (`aardvark-dns`, sur l'adresse de la passerelle `10.89.0.1`). Tout conteneur lancé avec `--network stack` peut joindre les autres **par leur nom de conteneur** : `api` résout vers l'IP d'`api`, `db` vers celle de `db`. Le `/etc/resolv.conf` du conteneur pointe vers `10.89.0.1`, avec `search dns.podman`.

Docker fonctionne de la même façon : son bridge par défaut n'a pas de DNS, ses réseaux user-defined en ont un. Le réseau par défaut sert au test rapide ; dès que deux conteneurs doivent se parler, on crée un réseau.

Un conteneur peut appartenir à **plusieurs** réseaux (`podman network connect admin api`) et porter des **alias** (`--network-alias backend`) : l'API est alors joignable sous deux noms. Un réseau isole aussi : un conteneur du réseau `admin` ne résout pas `api` tant qu'`api` n'y est pas connectée.

PIÈGE
Le DNS résout un nom en IP **au moment de la connexion**. nginx, lui, résout les noms de ses `proxy_pass` **une fois, au démarrage**, et garde l'IP. Si vous recréez `api` et qu'elle obtient une autre adresse, nginx continue d'envoyer vers l'ancienne : `502 Bad Gateway`, `Host is unreachable` dans ses logs. Remède : redémarrer `web`, ou configurer nginx avec une directive `resolver` et une variable. Spring Boot n'a pas ce problème : son pool de connexions ouvre de nouvelles connexions, donc refait la résolution.

4. Publier un port

Dans un réseau bridge, un conteneur n'est joignable que par ses voisins. Pour qu'un client **extérieur** (votre navigateur, `curl` depuis WSL) l'atteigne, il faut **publier** un port : `-p 18080:80` demande au moteur d'écouter sur le port 18080 de l'hôte et de relayer vers le port 80 du conteneur.

Syntaxe	Effet
`-p 18080:80`	hôte `0.0.0.0:18080` → conteneur `80`
`-p 127.0.0.1:18080:80`	seulement depuis la machine elle-même
`-P`	publie tous les ports `EXPOSE` sur des ports hôte aléatoires (`39201`…)
`EXPOSE 8080` (Dockerfile)	**ne publie rien** : documentation, lue par `-P` et par `podman ps`

`EXPOSE` est le malentendu classique : l'image dit « j'écoute sur 8080 », rien de plus. Sans `-p`, `curl localhost:8080` depuis l'hôte échoue, alors que les conteneurs du même réseau joignent le 8080 sans aucune publication. **Entre conteneurs, on ne publie pas** ; on publie pour l'extérieur.

> **→ SÉCURITÉ**
> Sur Linux, les ports **inférieurs à 1024** sont réservés à root. En rootless, `-p 80:80` échoue : `rootlessport cannot expose privileged port 80 … bind: permission denied` (ou `pasta failed` sur le réseau par défaut), et le conteneur reste en état `Created`. Trois issues : publier sur un port haut (`-p 8080:80`), abaisser la limite (`sysctl net.ipv4.ip_unprivileged_port_start=80`, une décision d'administrateur), ou mettre un reverse proxy root devant. En entreprise, un conteneur n'écoute **jamais** sur 80 ou 443 directement : c'est le rôle de l'équilibreur de charge ou de l'Ingress.

> **→ WINDOWS / WSL**
> Un port publié dans WSL est joignable depuis Windows sur `http://localhost:18080` : WSL 2 transmet automatiquement les ports écoutés dans la distribution vers `localhost` côté Windows — même avec `-p 127.0.0.1:…`. En revanche, les autres machines du réseau ne voient pas ce port, sauf à activer le mode réseau *mirrored* (`.wslconfig`, `networkingMode=mirrored`) ou une règle `netsh portproxy`.

5. host, none, et joindre l'hôte

`--network host` supprime l'isolation réseau : le conteneur utilise directement la pile de l'hôte, sans `-p` (ils n'ont plus de sens). C'est utile pour un outil de diagnostic réseau ; c'est dangereux pour un service, et en rootless, un nginx en `--network host` échoue à ouvrir le port 80 : `bind() to 0.0.0.0:80 failed (13: Permission denied)`.

`--network none` ne donne que `lo` : aucun accès réseau, ni entrant ni sortant. C'est le bon réglage pour un traitement par lots qui ne doit parler à personne, ou pour tester qu'une application n'a pas de dépendance réseau cachée.

Enfin, un conteneur a parfois besoin de joindre un service **de l'hôte** (une base installée dans WSL, un autre conteneur publié). Le nom `host.containers.internal` désigne l'hôte depuis le réseau par défaut — l'équivalent du `host.docker.internal` de Docker Desktop.

6. Les pods

PODMAN
Un **pod** est un groupe de conteneurs qui partagent **le même network namespace** : la même IP, les mêmes ports, le même `localhost`. Dans un pod, nginx joint l'API sur `localhost:8080`, et `-p` se déclare **sur le pod**, pas sur les conteneurs. C'est exactement le modèle de Kubernetes, dont Podman reprend le vocabulaire : `podman pod create --name stackpod -p 18090:80`, puis `podman run --pod stackpod …`. Un conteneur *infra* invisible tient le namespace ouvert. Docker n'a pas de pods ; `--network container:autre` en est l'approximation.

Retenez la nuance : dans un **réseau**, les conteneurs se parlent par leur nom ; dans un **pod**, par `localhost`. Les deux modèles coexistent, et la partie Kubernetes de cette série reposera sur le second.

7. En entreprise

- **Un réseau par application**, trois services dessus : `web`, `api`, `db`. Seul `web` publie un port. `db` n'est **jamais** publié en production ; en développement, `-p 127.0.0.1:5432:5432` pour y brancher un client SQL.
- **L'API trouve la base par son nom** : `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app`. Une URL avec `localhost` est le signe d'une configuration de poste copiée telle quelle dans un conteneur.
- **nginx fait reverse proxy** : `/` sert le front, `/api/` relaie vers `api:8080`. Le navigateur ne voit qu'une origine, ce qui évite le CORS, et l'API n'est pas exposée directement.

→ HTTP
Le **CORS** (*cross-origin resource sharing*) est une règle des navigateurs : une page servie par `localhost:18080` ne peut pas appeler `localhost:8080` sans que l'API y consente par des en-têtes. En plaçant le front et l'API derrière le même nginx, sur la même origine, le problème disparaît — c'est une des raisons d'être du reverse proxy, avec le TLS et la mise en cache.

→ SPRING BOOT
Le pool de connexions (HikariCP) ouvre quelques connexions au démarrage et en recrée au fil du temps. Si `db` n'est pas encore prête, Spring Boot échoue au démarrage : `Connection refused`. D'où l'importance de l'ordre de démarrage et des *healthchecks* (labo 09), et de l'option `spring.datasource.hikari.initializationFailTimeout` pour tolérer une base en retard.

À retenir

- `localhost` dans un conteneur, c'est le conteneur : on s'adresse aux autres par leur **nom**.
- Le réseau par défaut (pasta en rootless) et le réseau `podman` n'ont pas de DNS ; un réseau **user-defined** en a un : une application = un réseau.
- `-p hôte:conteneur` publie pour l'extérieur ; entre conteneurs du même réseau, rien à publier. `EXPOSE` ne publie rien.
- En rootless, pas de port < 1024 : `-p 8080:80`, ou un reverse proxy devant.
- nginx fige les IP de ses `proxy_pass` au démarrage : conteneur recréé = `502`, redémarrer nginx.
- `--network host` retire l'isolation, `--network none` retire le réseau, `host.containers.internal` désigne l'hôte.

- Un **pod** partage le network namespace : `localhost` entre conteneurs, `-p` sur le pod — le modèle de Kubernetes.

Vocabulaire

network namespace : pile réseau privée d'un conteneur. — **bridge** : commutateur virtuel reliant des conteneurs. — **veth** : câble virtuel conteneur ↔ bridge. — **NAT** : réécriture d'adresse pour sortir par l'hôte. — **pasta** : réseau rootless par défaut de Podman, copie l'adresse de l'hôte. — **rootlessport** : relais des ports publiés en rootless. — **aardvark-dns** : DNS interne des réseaux user-defined. — **publier (`-p`)** : relayer un port de l'hôte vers le conteneur. — **`EXPOSE`** : déclaration documentaire d'un port. — **alias** : nom supplémentaire d'un conteneur sur un réseau. — **pod** : groupe de conteneurs partageant le network namespace. — **reverse proxy** : serveur qui relaie les requêtes vers des services internes.